

个人博客系统详细设计报告

一、业务功能模块设计

我们将系统功能划分为前台（面向访客）和后台（面向博主/管理员），确保职责清晰，用户体验流畅。

1. 前台功能模块 (面向访客)

1.1. 文章浏览

1.1.1. 首页文章列表： 分页展示已发布的文章，显示标题、摘要、封面图、发布时间、所属分类、阅读量和评论数。

1.1.2. 文章详情页： 展示文章的完整内容（Markdown渲染）、标题、作者、发布时间、分类、标签、阅读量和评论数。

1.1.3. 文章归档： 按时间（年/月）归档文章，方便用户追溯历史内容。

1.1.4. 上一篇/下一篇： 在文章详情页提供导航，方便用户连续阅读。

1.2. 内容检索

1.2.5. 关键词搜索： 支持根据文章标题和内容进行模糊搜索。

1.2.6. 分类筛选： 点击分类标签，可查看该分类下的所有文章。

1.2.7. 标签筛选： 点击文章标签，可查看带有该标签的所有文章。

1.3. 互动评论

1.3.8. 发表评论： 访客可在文章详情页发表评论（需填写昵称、邮箱和内容）。

1.3.9. 评论列表： 展示文章的所有已审核通过的评论，支持嵌套回复展示。

1.4. 关于我

1.4.10. 个人简介页： 展示博主的头像、昵称、个人简介、技能栈、联系方式等。

2. 后台功能模块 (面向管理员)

2.1. 用户认证

2.1.1. 管理员登录： 通过用户名和密码登录后台管理系统。

2.1.2. JWT 令牌管理： 登录成功后颁发 JWT 令牌，后续请求需在 Header 中携带该令牌进行身份验证。

2.2. 仪表盘

2.2.3. 数据概览： 展示博客的核心数据，如文章总数、评论总数、总阅读量、最近发布的文章等。

2.3. 文章管理

2.3.4. 文章列表： 以表格形式展示所有文章，支持按标题搜索、按状态（草稿/已发布）筛选。

2.3.5. 新建/编辑文章： 集成 Markdown 编辑器，支持文章标题、摘要、封面图、分类、标签的设置，以及草稿保存和发布功能。

2.3.6. 删除文章： 支持单篇或批量删除文章（建议采用逻辑删除）。

2.4. 分类与标签管理

2.4.7. 分类管理： 对文章分类进行增、删、改、查操作。

2.4.8. 标签管理： 对文章标签进行增、删、改、查操作。

2.5. 评论管理

2.5.9. 评论列表： 展示所有评论，包括评论者信息、评论内容、所属文章等。

2.5.10. 评论审核： 对访客发表的评论进行审核（通过/拒绝）。

2.5.11. 评论删除： 删除违规或垃圾评论。

2.6. 系统设置

2.6.12. 博客信息配置： 可修改博客标题、副标题、SEO 关键词、页脚信息等。

2.6.13. 管理员信息修改： 修改管理员的个人资料和登录密码。

二、数据库表设计

我们将使用 MySQL 作为主数据库，以下是核心数据表的详细设计。所有表均采用`InnoDB`引擎和`utf8mb4`字符集。

1. 用户表(`users`) 用于存储管理员信息。

2. ```sql CREATE TABLE `users` (`id` INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY COMMENT '主键 ID', `username` VARCHAR(50) NOT NULL UNIQUE COMMENT '登录用户名', `password` VARCHAR(255) NOT NULL COMMENT '登录密码(加密存储)', `nickname` VARCHAR(50) NOT NULL COMMENT '显示昵称', `avatar` VARCHAR(255) DEFAULT NULL COMMENT '头像 URL', `email` VARCHAR(100) DEFAULT NULL COMMENT '邮箱', `bio` TEXT DEFAULT NULL COMMENT '个人简介', `created\_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间', `updated\_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间') ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='用户表'; ```

3. 文章表(`articles`) 博客的核心内容表。

4. ```sql CREATE TABLE `articles` (`id` INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY COMMENT '主键 ID', `user\_id` INT UNSIGNED NOT NULL COMMENT '作者 ID', `category\_id` INT UNSIGNED DEFAULT NULL COMMENT '分类 ID', `title` VARCHAR(255) NOT NULL COMMENT '文章标题',

```

        `slug` VARCHAR(255) NOT NULL UNIQUE
COMMENT '文章 URL 标识',        `summary`
VARCHAR(500) DEFAULT NULL COMMENT '文章摘要',
        `content` MEDIUMTEXT NOT NULL COMMENT '文章
内容(Markdown 格式)',        `cover_image` VARCHAR(255)
DEFAULT NULL COMMENT '封面图 URL',        `status`
TINYINT(1) NOT NULL DEFAULT 0 COMMENT '状态: 0-
草稿, 1-已发布',        `view_count` INT UNSIGNED NOT
NULL DEFAULT 0 COMMENT '阅读量',        `is_top`
TINYINT(1) NOT NULL DEFAULT 0 COMMENT '是否置顶:
0- 否, 1- 是',        `created_at` TIMESTAMP NOT NULL
DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
        `updated_at` TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP        ON        UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
`published_at` TIMESTAMP NULL DEFAULT NULL
COMMENT '发布时间',        FOREIGN KEY (`user_id`)
REFERENCES `users`(`id`) ON DELETE CASCADE,
        FOREIGN KEY (`category_id`) REFERENCES
`categories`(`id`) ON DELETE SET NULL,        INDEX
`idx_status` (`status`),        INDEX `idx_created_at`
(`created_at`)        ) ENGINE=InnoDB DEFAULT
CHARSET=utf8mb4 COMMENT='文章表';    ``

```

5. 分类表(`categories`) 用于对文章进行归类。

6. ```sql CREATE TABLE `categories` (`id` INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY COMMENT '主键 ID', `name` VARCHAR(50) NOT NULL UNIQUE COMMENT '分类名称', `slug` VARCHAR(50) NOT NULL UNIQUE COMMENT '分类 URL 标识', `description` VARCHAR(255) DEFAULT NULL COMMENT '分类描述', `created\_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间', `updated\_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间') ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='分类表'; ```

7. 标签表(`tags`) 用于为文章添加关键词。

8. ```sql CREATE TABLE `tags` (`id` INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY COMMENT '主键 ID', `name` VARCHAR(50) NOT NULL UNIQUE COMMENT '标签名称', `slug` VARCHAR(50) NOT NULL UNIQUE COMMENT '标签 URL 标识', `created\_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间') ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='标签表'; ```

9. 文章-标签关联表(`article\_tags`) 解决文章与标签的多对多关系。

```
10.``sql      CREATE TABLE `article_tags` (          `article_id`
INT UNSIGNED NOT NULL COMMENT '文章 ID',
`tag_id` INT UNSIGNED NOT NULL COMMENT '标签 ID',
PRIMARY KEY (`article_id`,`tag_id`),          FOREIGN
KEY (`article_id`) REFERENCES `articles`(`id`) ON DELETE
CASCADE,          FOREIGN KEY (`tag_id`) REFERENCES
`tags`(`id`) ON DELETE CASCADE          ) ENGINE=InnoDB
DEFAULT CHARSET=utf8mb4 COMMENT='文章-标签关
联表';    ``
```

11. 评论表(`comments`) 存储用户对文章的评论。

```
12.``sql      CREATE TABLE `comments` (          `id` INT
UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY
KEY COMMENT '主键 ID',          `article_id` INT
UNSIGNED NOT NULL COMMENT '文章 ID',
`parent_id` INT UNSIGNED DEFAULT NULL COMMENT '
父评论 ID(用于嵌套回复)',          `nickname` VARCHAR(50)
NOT NULL COMMENT '评论者昵称',          `email`
VARCHAR(100) NOT NULL COMMENT '评论者邮箱',
`content` TEXT NOT NULL COMMENT '评论内容',
`status` TINYINT(1) NOT NULL DEFAULT 0 COMMENT '状
态: 0-待审核, 1-已通过, 2-已拒绝',          `created_at`
TIMESTAMP          NOT          NULL          DEFAULT
```

```

CURRENT_TIMESTAMP COMMENT '创建时间',
FOREIGN KEY (`article_id`) REFERENCES `articles`(`id`)
ON DELETE CASCADE, FOREIGN KEY (`parent_id`)
REFERENCES `comments`(`id`) ON DELETE CASCADE,
INDEX `idx_article_status` (`article_id`, `status`)
)
ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COMMENT='评论表';

```

13. 系统配置表 (`configs`) 存储博客的全局配置信息。

```

14. ``sql CREATE TABLE `configs` (
    `id` INT
    UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY
    KEY COMMENT '主键 ID',
    `config_key`
    VARCHAR(50) NOT NULL UNIQUE COMMENT '配置键',
    `config_value` TEXT NOT NULL COMMENT '配置值',
    `description` VARCHAR(255) DEFAULT NULL
    COMMENT '配置描述',
    `updated_at` TIMESTAMP
    NOT NULL DEFAULT CURRENT_TIMESTAMP ON
    UPDATE CURRENT_TIMESTAMP COMMENT '更新时间'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COMMENT='系统配置表';

```

三、RESTful API 接口设计

我们将遵循 RESTful 风格设计后端 API，所有接口返回的数据格式统一为 JSON。

通用响应格式

```
{
```

```
"code": 200, // 业务状态码, 200 表示成功
"message": "操作成功", // 提示信息
"data": { ... } // 响应数据
}
```

1. 认证接口 (/api/auth)

1.1. POST /api/auth/login: 管理员登录

1.1.1. 请求体: `{ "username": "admin", "password": "password123" }`

1.1.2. 响应: `{ "code": 200, "data": { "token": "jwt_token_string" } }`

2. 文章接口 (/api/articles)

2.1. GET /api/articles: 获取文章列表

2.1.1. 查询参数: `?page=1&limit=10&keyword=xxx&category=tech`

2.2. GET /api/articles/:slug: 获取单篇文章详情

2.3. POST /api/articles: 创建文章 (需认证)

2.3.2. 请求体: `{ "title": "...", "content": "...", "categoryId": 1, "tags": [1, 2] }`

2.4. PUT /api/articles/:id: 更新文章 (需认证)

2.5. DELETE /api/articles/:id: 删除文章 (需认证)

3. 分类接口 (/api/categories)

3.1. GET /api/categories: 获取所有分类

3.2. POST /api/categories: 创建分类 (需认证)

3.3. PUT /api/categories/:id: 更新分类 (需认证)

- 3.4. `DELETE /api/categories/:id`: 删除分类 (需认证)
- 4. 标签接口 (`/api/tags`)
 - 4.1. `GET /api/tags`: 获取所有标签
 - 4.2. `POST /api/tags`: 创建标签 (需认证)
 - 4.3. `PUT /api/tags/:id`: 更新标签 (需认证)
 - 4.4. `DELETE /api/tags/:id`: 删除标签 (需认证)
- 5. 评论接口 (`/api/comments`)
 - 5.1. `GET /api/comments`: 获取评论列表 (通常按文章 ID 筛选)
 - 5.1.1. 查询参数: `?articleId=1&status=1`
 - 5.2. `POST /api/comments`: 发表评论
 - 5.2.2. 请求体: `{ "articleId": 1, "nickname": "访客", "email": "a@b.com", "content": "..." }`
 - 5.3. `PUT /api/comments/:id/status`: 审核评论 (需认证)
 - 5.3.3. 请求体: `{ "status": 1 }`
 - 5.4. `DELETE /api/comments/:id`: 删除评论 (需认证)
- 6. 系统配置接口 (`/api/configs`)
 - 6.1. `GET /api/configs`: 获取所有公开配置 (用于前台)
 - 6.2. `PUT /api/configs`: 更新配置 (需认证)

四、Redis 应用场景

Redis 在本系统中扮演着提升性能和保障安全的关键角色。

1. 数据缓存

- 1.1. 缓存键示例: ``blog:article:list:1`` (存储第一页的文章列表 JSON)
 - 1.2. 缓存键示例: ``blog:config:all`` (存储所有系统配置)
 - 1.3. 策略: 在读取文章列表、系统配置等高频读取但低频修改的数据时, 优先从 Redis 中获取。如果缓存未命中, 则查询 MySQL, 并将结果写入 Redis, 同时设置一个合理的过期时间 (如 10 分钟)。
2. JWT 令牌管理
 - 2.1. 黑名单机制: 当用户退出登录时, 将其 JWT 令牌的 ``jti`` (唯一标识) 存入 Redis, 并设置过期时间为令牌的剩余有效期。在认证中间件中, 除了验证令牌签名, 还需检查该令牌是否在 Redis 黑名单中。
3. API 限流
 - 3.1. 限流键示例: ``ratelimit:login:192.168.1.1`` (记录某 IP 的登录尝试次数)
 - 3.2. 策略: 在登录等敏感接口, 使用 Redis 的 ``INCR`` 和 ``EXPIRE`` 命令实现滑动窗口限流。例如, 限制同一 IP 在 1 分钟内最多尝试登录 5 次, 超过则拒绝服务。

五、项目目录结构参考

- 后端 (backend/)
- ```
`` backend/ |----- src/ | |----- config/
数据库、Redis 等配置 | |----- controllers/ # 控
制器, 处理业务逻辑 | |----- models/ #
```

```

Sequelize 模型定义 | |—— routes/ # API 路由
 | |—— middlewares/ # 认证、错误处理等中间件
 | |—— utils/ # 工具函数 (如 JWT 生成)
 | |—— app.js # Express 应用入口
 |—— .env # 环境变量
 |—— server.js # 服务器启动文件
 |—— package.json

```

- 前端 (frontend/)

```

• `` frontend/ |—— src/ | |——
components/ # 通用组件 (Button, Input 等) |
|—— pages/ # 页面组件 (Home, Article, Admin 等)
| |—— services/ # API 请求封装
|—— stores/ # 状态管理 (Zustand/Redux)
|—— utils/ # 工具函数
根组件 | |—— main.jsx # 应用入口
|—— .env # 环境变量
|—— vite.config.js # Vite 配置文件

```

(AI 生成)